# Lab Reflection — WalkMates

**Lab:** 2 **Pair:** Lena Edlind, Jakob Cederblad **Repo commit/tag:** https://github.com/leed2400/walkmates-test / 142ab10171a8e7bfea166761abf4092be9b88d33

## 1. What we did

In Part A, we measured baseline coverage for `PricingCalculator` using JaCoCo and added structural unit tests for null guards, free shelter listings, and overnight bookings to achieve 100% branch and statement coverage. We then wrote a boundary test for exactly 480 minutes, observed the expected failure, and corrected the comparison operator from `>=` to `>` in `PricingCalculator`. We also corrected the phone validation regex in `Seeker.java` ( `\d{7}` to `\d{8}` ); this production fault was originally discovered during test execution in Lab 1, and we implemented the fix here to ensure a green test suite before running PIT.

In Part B, we ran PIT mutation testing to evaluate our test strength and find surviving mutants. We then used Mockito to write isolated component tests for `SeekerService` (testing payment success, decline, and timeout) and `BookingService` (testing confirmation notifications and active booking limits). We fixed the booking-limit comparison fault identified in Lab 1. Finally, we performed a regression selection and prioritization analysis for the `feature/weekend-surcharge` scenario.

## 2. What we found

We found that structural coverage can be misleading. Our tests reached 100% branch and line coverage on `PricingCalculator`, yet completely missed the boundary bug on line 49 because our test only executed the code with 600 minutes instead of checking the exact 480-minute threshold.

We also saw how mutation testing catches what code coverage misses. PIT flagged a surviving conditional boundary mutant on line 71 in `BookingService`. By adding an isolated unit test for a seeker who had already reached their limit, we reproduced the failure against `1 > 1` and fixed it to `>=`. After adding the Mockito tests, our project mutation score increased from 35% to 55%, and `PricingCalculator` reached 100% mutation coverage.

## 3. AI use (be honest — it doesn't lower your grade)

We did not use AI in Part A. In Part B, we used the AI to discuss our ideas and check our understanding. We asked the AI to explain the outputs of the JaCoCo and PIT reports so we could cross-check that our own interpretations of the missed branches and surviving mutants were correct. We also used the AI to bounce ideas regarding which tests to prioritize in the regression selection analysis to ensure our reasoning about execution order and test isolation made sense.

## 4. Judgment

We had to apply our own judgment in several key areas where tools and AI fell short:

- **Coverage vs. Correctness:** JaCoCo showed 100% branch coverage and marked the code completely green. We had to recognize from the specification that 480 minutes was an off-by-one boundary that required its own dedicated test case.
- **Validating Existing Code:** When the phone validation test failed, we had to evaluate the requirement specification (FR-1.1) and decide that the production code was the faulty part, rather than just changing the test data to make the suite green.
- **Regression Selection Trade-offs:** For the regression analysis in Activity 4.3, tools cannot automatically know the business impact of a change. We had to decide which tests were change-relevant (`PricingCalculatorStructuralTest` and `BookingServiceTest`) and argue why running the slow Spring Boot web tests (`MatchControllerWebTest`) would be wasteful.

## 5. What we'd test next

If we had another hour, we would write unit tests for the remaining rejection rules in `BookingService`: testing that an unavailable listing is rejected (Rule 1), that a provider at full capacity rejects new bookings (Rule 3), and that a seeker with insufficient balance cannot book (Rule 5). We would also add tests for `SeekerService.register()` to test duplicate email rejection.